Silvestre Ronin, silver3@rpi.edu, 662043925
Kurbiel Chris, kurbic@rpi.edu, 662041666
Homework 4
Professor Radev
COGNITIVE MODELING - COGS 4210 - 01
April 15, 2026

The link to the github repo for this assignment can be found here.

1 Problem 1 [FINISHED]

All statements that are false: 3,4,6,7,8,10.

Assumption on 8: From week7 slides page24, while it is true that LPD is approximated by evaluating likelihood of each posterior draw and then taking the average of all resulting likelihood values, the statement does not mention taking the log, which is in the equation. Hence, this statement is incomplete and considered to be false.

2 Problem 2 [FINISHED]

There are 3 files associated with this question that can be found on the Github repo. The description and links to each file is found below:

- The One-High-Threshold Model (1HT) stan file can be found here.
- The Two-High-Threshold Model (2HT) stan file can be found here.
- The link to the interactive python notebook file that implements both stan models can be found here.

OUTPUT

Estimation summary, convergence, and efficiency diagnostics of 1HT model:

	Mean	MCSE	StdDev	MAD	5%	50%	95%	ESS_bulk	ESS_tail	ESS_bulk/s	R_hat
lp__	-24.271000	0.018564	1.092210	0.752007	-26.466000	-23.931600	-23.257400	4197.16	4221.60	10572.2	1.00054
d	0.663350	0.001729	0.123184	0.120138	0.442232	0.675420	0.842201	5313.03	4863.94	13382.9	1.00075
g	0.190084	0.001112	0.083458	0.083065	0.071823	0.180136	0.344228	5435.90	5153.61	13692.5	1.00025
old_tree[1]	0.730623	0.001160	0.092833	0.094209	0.567053	0.738219	0.871833	6355.58	5941.96	16009.0	1.00060
old_tree[2]	0.269377	0.001160	0.092833	0.094209	0.128167	0.261781	0.432947	6355.58	5941.96	16009.0	1.00060
new_tree[1]	0.190084	0.001112	0.083458	0.083065	0.071823	0.180136	0.344228	5435.90	5153.61	13692.5	1.00025
new_tree[2]	0.809916	0.001112	0.083458	0.083065	0.655772	0.819864	0.928177	5435.90	5153.61	13692.5	1.00025
pred_y_old[1]	14.642100	0.030624	2.707830	2.965200	10.000000	15.000000	19.000000	7854.56	8277.66	19784.8	1.00066
pred_y_old[2]	5.357900	0.030624	2.707830	2.965200	1.000000	5.000000	10.000000	7854.56	8277.66	19784.8	1.00066
pred_y_new[1]	3.812600	0.027924	2.370370	2.965200	0.000000	3.000000	8.000000	7178.37	8048.95	18081.5	1.00022
pred_y_new[2]	16.187400	0.027924	2.370370	2.965200	12.000000	17.000000	20.000000	7178.37	8048.95	18081.5	1.00022
log_likelihood	-4.009570	0.016128	1.005800	0.707981	-6.041650	-3.691570	-3.061510	4742.92	4886.03	11946.9	1.00048

Estimation summary, convergence, and efficiency diagnostics of 2HT model:

	Mean	MCSE	StdDev	MAD	5%	50%	95%	ESS_bulk	ESS_tail	ESS_bulk/s	R_hat
lp__	-23.624600	0.016952	1.052900	0.750901	-25.759300	-23.295700	-22.618000	4190.46	4817.66	9930.01	1.00071
d	0.578905	0.001333	0.120722	0.120699	0.364902	0.587859	0.760804	8573.69	5798.36	20316.80	1.00024
g	0.394747	0.001501	0.137420	0.145784	0.177960	0.390973	0.627264	8206.26	5707.39	19446.10	1.00088
old_tree[1]	0.745992	0.000942	0.090664	0.090732	0.582318	0.754099	0.879806	9312.92	7098.39	22068.50	1.00044
old_tree[2]	0.254008	0.000942	0.090664	0.090732	0.120194	0.245901	0.417682	9312.92	7098.39	22068.50	1.00044
new_tree[1]	0.167087	0.000877	0.077575	0.076886	0.059605	0.156413	0.312331	7683.07	6051.39	18206.30	1.00081
new_tree[2]	0.832913	0.000877	0.077575	0.076886	0.687669	0.843587	0.940395	7683.07	6051.39	18206.30	1.00081
pred_y_old[1]	14.919800	0.026926	2.636410	2.965200	10.000000	15.000000	19.000000	9597.02	9093.61	22741.70	1.00068
pred_y_old[2]	5.080200	0.026926	2.636410	2.965200	1.000000	5.000000	10.000000	9597.02	9093.61	22741.70	1.00068
pred_y_new[1]	3.353300	0.024116	2.264460	1.482600	0.000000	3.000000	8.000000	8875.88	9065.32	21032.90	1.00052
pred_y_new[2]	16.646700	0.024116	2.264460	1.482600	12.000000	17.000000	20.000000	8875.88	9065.32	21032.90	1.00052
log_likelihood	-3.926960	0.014507	0.906173	0.645108	-5.738840	-3.647340	-3.059740	4284.99	5221.77	10154.00	1.00082

ANALYSIS

First, it is worth noting that we set $d_{\text{true}} = 0.7, g_{\text{true}} = 0.3$. We observe that both the 1HT and 2HT models appeared to have converged well. From the $\hat{R}$ values for both models, it is roughly 1 across each parameter, suggesting overall that the MCMC sampling is both

quite efficient and well-converged. Additionally, the effective sample size (ESS) is large for both models, roughly around 10000. This suggests that the models' posterior draws are of high quality and independent, and thus explore the posterior distribution efficiently.

Specifically looking at the 1HT model, the posterior mean estimates are $d = 0.66, g = 0.19$. Specifically looking at the 2HT model, the posterior mean estimates are $d = 0.58, g = 0.39$. Indeed, we observe that the models suggest different estimates for the two key parameters. The 1HT model recovers d closer than the 2HT model, and in contrast, the 2HT model recovers g closer than the 1HT model. As visualised in slide12 from the slides, the old trees are the same, but the new tree for 2HT adds an additional detection mechanism, while the 1HT is only dependent on guessing (only parameter g is present in the new tree for 1HT). Hence, the difference in recovered values between the two models may be from the fact that there is structurally different detection and guessing mechanisms for both trees. Yet, the two models are able to rather faithfully recover the parameters, as they are not far off from the true values.

3 Problem 3 [FINISHED]

There are 2 files associated with this question that can be found on the Github repo. The description and links to each file is found below:

- The stan file can be found here.
- The link to the interactive python notebook file that implements both stan models can be found here.

OUTPUT

	Mean	MCSE	StdDev	MAD	5%	50%	95%	ESS_bulk	ESS_tail	ESS_bulk/s	R_hat
lp__	-2995.630000	0.020425	1.400900	1.209730	-2998.340000	-2995.310000	-2994.000000	4711.82	6618.93	204.568	1.00036
beta_0	-0.421224	0.000329	0.032335	0.032438	-0.473982	-0.421309	-0.368776	9657.20	7733.48	419.277	1.00105
beta[1]	1.005060	0.000469	0.045233	0.045643	0.931646	1.004380	1.079720	9317.71	7475.44	404.538	1.00039
beta[2]	0.371293	0.000475	0.045723	0.045723	0.295877	0.371574	0.445694	9294.12	7526.06	403.513	1.00004
beta[3]	0.489026	0.000424	0.041288	0.042168	0.421170	0.489065	0.557374	9519.99	7543.43	413.320	1.00023

SUPPLEMENTARY OUTPUT

Note: I didn't read question 4 until after finishing this question. So initially, very confused on the purpose of the train-test split, so I decided to calculate the accuracy on the test set. This result is included here as a supplement, following the math derived for the Bayesian regression formula used for this question which can be found in the interactive python notebook link at the top of this question's writeup.

```python
# obtaining the beta means...
beta_means = fit.summary().loc[['beta[1]', 'beta[2]', 'beta[3]'], 'Mean'].values
beta_0_mean = fit.summary().loc['beta_0', 'Mean']

# using the inverse logit (sigmoid) function to get p_n...
from scipy.special import expit
logits = beta_0_mean + X_test_standardised @ beta_means
test_probs = expit(logits)

# converting probabilities to 0 or 1 predictions (threshold at 0.5)...
predictions = (test_probs > 0.5).astype(int)

# comparing to the test set labels...
accuracy = (predictions == y_test).mean()
print(f"Test Set Accuracy: {accuracy:.2%}")
```

✓ 0.6s

```
Test Set Accuracy: 75.28%
```

ANALYSIS

Our decision variable is binary, so hence, the appropiate likelihood is Bernoulli. That is, for sample n, we have that $y_n \sim \text{Bernoulli}(p_n)$.

Particularly, our Bayesian linear regression probability for sample n is formulated as follows:

$$\text{logit}(p_n) = \beta_0 + \beta_{\text{attr}} x_{\text{attr},n} + \beta_{\text{fun}} x_{\text{fun},n} + \beta_{\text{shar}} x_{\text{shar},n},$$

where $\beta_0, \beta_{\text{attr}}, \beta_{\text{fun}}, \beta_{\text{shar}} \sim \mathcal{N}(0, 2.5)$.

Then, we obtain p_n itself through the sigmoid function, which is

$$\frac{1}{1+\exp^{-(\beta_0+\beta_{\text{attr}}x_{\text{attr},n}+\beta_{\text{fun}}x_{\text{fun},n}+\beta_{\text{shar}}x_{\text{shar},n})}}.$$

From the $\hat{R}$ values for the model, it is roughly 1 across each parameter, suggesting overall that the MCMC sampling is both quite efficient and well-converged. Additionally, the effective sample size (ESS) is large for the model, roughly around 10000. This suggests that the models' posterior draws are of high quality and independent, and thus explore the posterior distribution efficiently. It is worth noting that in the data preprocessing stage, the attributes we chose are attractiveness, fun, and shared interest.

Intuitively, meeting someone for only 4 minutes, the first impression matters, and a participant would mainly judge if he/she wants to see the other person based on "easily observable" traits. We report the posterior (β_i for $i = 0, \cdots, 3$ as follows: $\beta_0 = -.42, \beta_{\text{attractivenes}} := \beta_1 = 1.00, \beta_{\text{fun}} := \beta_2 = 0.37, \beta_{\text{shared interest}} := \beta_3 = 0.49$. We observe that the highest posterior is $\beta_{\text{attractivenes}}$ with a value of 1, implying that out of feature selection, attractiveness was the main factor. The least factor out of feature selection is fun. Intuitively, this makes sense also, since the first impression you have of someone is his/her apperance. Even not in speed dating, there exists such thing called the "halo effect", where there is tendency to assume that attractive people are more likely to possess favorably personality traits (Wikipedia).

The credible interval is reported as follows: $\beta_0 : [-0.47, -0.37], \beta_1 : [0.93, 1.08], \beta_2 : [0.29, 0.45], \beta_3 : [0.42, 0.56]$.

Answering the 3 questions

- Yes, model did converge as discussed above with $\hat{R} \approx 1.00$.
- Attractiveness, as discussed above.
- We observe from supplementary output that test set accuracy is 75%. It is not bad but perhaps could be improved upon by better model. One such way is perhaps by implementing a hierarchial model as talked about in class instead. The current model assumes everyone is the "same" when making these decisions on his/her dating life, since there is "global" posteriors. Implementing individual $\beta_i^{(n)}$ for each n participant may allow for a better output.

Alternatively, we could use scikit-learn BayesianRidge. However, it is "less appropiate" since is designed for continuous outcomes and assumes a normal likelihood rather than Bernoulli, and hence, may produce output y that is outside the range $[0, 1]$.

4 Problem 4 [NEEDS WRITEUP]

There are 2 files associated with this question that can be found on the Github repo. The description and links to each file is found below:

- The stan file can be found here.
- The link to the interactive python notebook file that implements both stan models can be found here.

OUTPUT

	Mean	MCSE	StdDev	MAD	5%	50%	95%	ESS_bulk	ESS_tail	ESS_bulk/s	R_hat
lp__	-2995.640000	0.020004	1.389110	1.200610	-2998.320000	-2995.330000	-2994.020000	5169.16	6286.07	86.8869	1.00117
beta_0	-0.421455	0.000327	0.032524	0.032869	-0.475265	-0.421617	-0.367242	9951.43	6756.13	167.2710	1.00019
beta[1]	1.006100	0.000498	0.045679	0.046282	0.931249	1.005770	1.080630	8510.72	6974.66	143.0540	1.00079
beta[2]	0.370519	0.000483	0.045609	0.045686	0.293561	0.370111	0.445261	8927.14	7429.97	150.0540	1.00026
beta[3]	0.488746	0.000425	0.041186	0.041719	0.421526	0.488760	0.556449	9415.00	7620.20	158.2540	1.00034
...	...	...	...	...	...	...	...	...	...	...	...
p_hat[1448]	0.543328	0.000136	0.013498	0.013478	0.521231	0.543185	0.565578	9856.80	7913.12	165.6800	1.00029
p_hat[1449]	0.078978	0.000069	0.006287	0.006356	0.069065	0.078739	0.089626	8323.95	7089.44	139.9150	1.00030
p_hat[1450]	0.670160	0.000118	0.012283	0.012301	0.649992	0.670265	0.690225	10925.30	8133.79	183.6400	1.00048
p_hat[1451]	0.594484	0.000163	0.015975	0.015745	0.568060	0.594454	0.620851	9646.41	7695.28	162.1440	1.00034
p_hat[1452]	0.097058	0.000080	0.007316	0.007343	0.085602	0.096849	0.109269	8329.83	6977.76	140.0140	1.00040

1457 rows × 11 columns

```python
# extracting the predicted probabilities for the test set from the fitted model...
p_hat_samples = fit.stan_variable("p_hat")

# calculating mean predicted probability...
p_bar = p_hat_samples.mean(axis=0)

# brier score (as taken from hw pdf)...
brier_score = ((p_bar - y_test.to_numpy())**2).mean()

# calculating accuracy on the test set...
predictions = (p_bar > 0.5).astype(int)
accuracy = (predictions == y_test.to_numpy()).mean()

print(f"Brier Score: {brier_score:.4f}")
print(f"Test Set Accuracy: {accuracy:.2%}")
```

```
Brier Score: 0.1684
Test Set Accuracy: 75.28%
```

ANALYSIS

We report Brier score of 0.17 and classification accuracy of 0.75. Compared to baseline, the classification accuracy is much higher, since 0.75 >> 0.50, indicating that our model is not just randomly guessing or outputting single value. When we reduce the full posterior

predictive distribution of $\hat{p}_m$ to a single mean $\bar{p}_m$, we cannot distinguish whether a prediction is stable across posterior or if it is highly sensitive to parameter uncertainty. Suppose we have two different test instances that have predictive mean of 0.5. In one instance, the model could be "confident unsure" if all the MCMC samples cluster at 0.5. In the other instance, the model might be "confused" with samples fluctuate between close to 0.1 and close to 0.9 (and hence, the average turns out to be 0.5). Then, we cannot tell whether the instance is stable or sensitive. We can use the individual MCMC samples to obtain a distribution over the test performance itself through calculating performance metric for every individual MCMC iteration instead of averaging. In this case, we use Brier score and classification accuracy. Interpreting the reported Brier score, it is 0.17 which is low, suggesting that the model assigns high probability to things that actually happen and low probability to things that don't.

5 Problem 5 [FINISHED]

Ronin idea:

The final project can be divided into 3 main phases, which we now discuss sequentially.

Phase 1: Collect data and formulate a model.

Particularly, I enjoyed working through the speed dating dataset and I think it is very interesting how my intuition did indeed align with the results. I am interested in perhaps taking larger subset (or even all the attributes) instead of just 3. This, would still use the Bernoulli likelihood, but with more posteriors to estimate. The necessary steps taken to expand on the data as well as the model would be taken.

Phase 2: Fit the model and diagnose.

We fit the model in Stan. Then, in addition to the test set accuracy and Brier score as shown here, we will try to simualte "fake" decisions for the test instances. For example, suppose that in the real dataset, there is 40% "yes" decisions. If the model simulates near this, then it would imply that hte model is indeed well-calibrated.

Phase 3: Ask LLM to be a scientist.

We now take the dataset, the model, and all outputs from phase2, and feed it to LLM. Now, we ask the LLM to improve on the current model. We then rerun phase2, reporting results and analysis LLM performance compared to our model.

The data and parameters are described in Phase1. Models implemented are Bernoulli and hierarchial, either of which is indeed adequate for our case. Computational faithfulness is ensured through analysis of the model fit from Stan (looking at $\hat{R}$ and ESS). Phase2 describes how we criticise the models, which is parameter estimation, prediction, and simulation.

Chris idea:

Weather-Themed Model Creation, Evaluation With Limited Features, & LLM Model Comparison We are considering creating a prediction model for the temperature in Troy NY for a given day in the next year. Our data will be the temperature air pressure, and precipitation data from every day for many years in Troy NY up through 2024 and see how accurately the model said the temperature was for any 2025 day (*since that ground truth is available, this will be our means to criticize the model*). We will do this for the temperature high, the low, the average temperature of the day (*computed as the sum of the hourly temperatures of that day*), barring that data is also available. While models already exist that consider many more factors than just prior temperature, pressure and precipitation, we are interested in seeing how accurate our model is through limiting the features. We intend to use Stan and experiment with various prior distributions and see

which yield the closest results to the true 2025 dataset. To ensure computational faithfulness, we will run the Stan diagnostics to verify the potential scale reduction factor ($\hat{R}$) ¡ 1.005, confirm the effective sample size bulk and tail values are less than 400 for the standard 4-chain procedure, and make sure there aren't any divergence warnings when the data is being sampled. Next, we will compare our model to LLM-generated Stan code and determine if that served as a better or worse predictor through the aforementioned evaluation criteria

We have stretch goals of also predicting the precipitation percentages for given days, the air pressure, and most ambitiously, try to create a new temperature predictive model using the deep learning resources Keras provides.

This website seems to hold a significant amount of Troy weather data, however we will continue looking for more complete datasets: usclimatedata.com